

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

React Router

Come creare pagine, menu, layout, pagina non trovata, navigazione da codice e parametri dinamici in React.



CAPITOLO 1

Che cos'è una SPA

NOTA

SPA significa Single Page Application: l'app carica una sola pagina HTML e aggiorna il contenuto nel browser.

In un sito tradizionale, visitare una nuova pagina richiede al browser di chiedere un nuovo documento al server. In una SPA il browser carica l'applicazione una volta sola; quando l'utente passa dalla Home ai Servizi, React sostituisce soltanto il contenuto necessario.

CAPITOLO 2

Cos'è React Router e come si installa

NOTA

React Router collega un percorso dell'URL, come `/contatti`, al componente React da mostrare.

React da solo aggiorna l'interfaccia, ma non decide quale pagina mostrare quando cambia l'indirizzo del browser. React Router aggiunge questa capacità: legge l'URL, trova la route corrispondente e mostra il componente previsto.

Nel progetto creato con Vite installiamo il pacchetto con pnpm. Poi avvolgiamo l'applicazione con **BrowserRouter** nel file di ingresso.

Esempio

```
pnpm add react-router
```

File `src/main.jsx`:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

CAPITOLO 3

Routes e Route

NOTA

Routes raccoglie le route dell'app.

Route associa una prop **path** a un componente tramite la prop **Component**.

Per il sito vetrina creiamo pagine minime: ognuna ha soltanto un titolo e un testo. La prop **path** indica l'URL da riconoscere; la prop **Component** riceve il nome del componente, senza parentesi graffe con JSX.

Esempio

File **src/App.jsx**:

```
import { Route, Routes } from "react-router";

function Home() {
  return <main><h1>Studio Aurora</h1><p>Idee digitali per piccole attività.</p></main>;
}

function ChiSiamo() {
  return <main><h1>Chi siamo</h1><p>Siamo un piccolo studio creativo.</p></main>;
}

function Contatti() {
  return <main><h1>Contatti</h1><p>Scrivici per conoscere il progetto.</p></main>;
}

function App() {
  return (
    <Routes>
      <Route path="/" Component={Home} />
      <Route path="/chi-siamo" Component={ChiSiamo} />
      <Route path="/contatti" Component={Contatti} />
    </Routes>
  );
}

export default App;
```

CAPITOLO 4

Link e NavLink

NOTA

Link cambia route senza ricaricare tutta la pagina.

NavLink è pensato per i menu e permette di riconoscere la route attiva.

Per navigare all'interno di una SPA non usiamo un tag `<a>` normale. Usiamo **Link** e la prop **to**. In questo modo React Router intercetta il click e aggiorna soltanto la parte necessaria dell'app.

NavLink funziona come **Link**, ma passa l'informazione **isActive** alla funzione di **className**. È utile per evidenziare la voce del menu della pagina aperta.

Esempio

```
import { Link, NavLink } from "react-router";

function Menu() {
  return (
    <nav>
      <NavLink to="/" end>Home</NavLink>
      <NavLink to="/chi-siamo">Chi siamo</NavLink>
      <NavLink
        to="/servizi"
        className={({ isActive }) => isActive ? "attivo" : ""}
      >
        Servizi
      </NavLink>
      <NavLink to="/contatti">Contatti</NavLink>

      <Link to="/servizi/web-design">Scopri il web design</Link>
    </nav>
  );
}
```

CAPITOLO 5

Layout e Outlet

NOTA

Un layout raccoglie le parti condivise, come il menu.

Outlet indica il punto in cui viene mostrata la route figlia.

Il menu del sito vetrina compare in ogni pagina. Per evitare di copiarlo nelle pagine Home, Chi siamo, Servizi e Contatti, lo spostiamo in un file separato chiamato `layouts/MainLayout.jsx`.

Le pagine diventano route figlie di `MainLayout`. La route con `index` è la pagina predefinita del layout e viene mostrata all'URL `/`.

Esempio

File `src/layouts/MainLayout.jsx`:

```
import { NavLink, Outlet } from "react-router";

function MainLayout() {
  return (
    <>
      <nav>
        <NavLink to="/" end>Home</NavLink>
        <NavLink to="/chi-siamo">Chi siamo</NavLink>
        <NavLink to="/servizi">Servizi</NavLink>
        <NavLink to="/contatti">Contatti</NavLink>
      </nav>
      <Outlet />
    </>
  );
}

export default MainLayout;
```

File `src/App.jsx`:

```
import { Route, Routes } from "react-router";
import MainLayout from "../layouts/MainLayout";

function App() {
  return (
    <Routes>
      <Route path="/" Component={MainLayout}>
        <Route index Component={Home} />
        <Route path="chi-siamo" Component={ChiSiamo} />
        <Route path="servizi" Component={Servizi} />
        <Route path="contatti" Component={Contatti} />
      </Route>
    </Routes>
  );
}
```

CAPITOLO 6

Pagina 404

NOTA

La route con `path="*"` viene usata quando nessun altro percorso corrisponde all'URL.

Un utente può digitare un indirizzo inesistente o aprire un vecchio collegamento. Senza una route di recupero, l'app non avrebbe un messaggio utile da mostrare. Una pagina 404 spiega il problema e offre un link per tornare alla Home.

Esempio

```
import { Link, Route } from "react-router";

function PaginaNonTrovata() {
  return (
    <main>
      <h1>Pagina non trovata</h1>
      <p>L'indirizzo visitato non esiste.</p>
      <Link to="/">Torna alla Home</Link>
    </main>
  );
}

<Route path="*" Component={PaginaNonTrovata} />
```

CAPITOLO 7

Navigare con useNavigate

NOTA

`useNavigate` restituisce una funzione per cambiare route da JavaScript.

`Link` viene usato quando la navigazione è già visibile nel JSX. `useNavigate` è utile quando la navigazione dipende da un'azione: per esempio, dopo l'invio di un form o dopo il click di un bottone.

Esempio

```
import { useNavigate } from "react-router";

function Home() {
  const navigate = useNavigate();

  function mostraServizi() {
    navigate("/servizi");
  }

  return (
    <main>
      <h1>Studio Aurora</h1>
      <p>Idee digitali per piccole attività.</p>
      <button onClick={mostraServizi}>Scopri i servizi</button>
    </main>
  );
}
```

CAPITOLO 8

Parametri dinamici e useParams

NOTA

Un segmento che inizia con `:` è un parametro dinamico.

`useParams` legge il valore presente nell'URL.

Non serve una route separata per ogni servizio. Il percorso `servizi/:serviceId` riconosce sia `/servizi/web-design` sia `/servizi/consulenza`. La parte variabile viene letta nel componente tramite `useParams`.

Nel sito vetrina i dati possono stare in un piccolo oggetto locale. La chiave dell'oggetto corrisponde al valore scritto nell'URL.

Esempio

```
import { Link, Route, useParams } from "react-router";

const servizi = {
  "web-design": {
    titolo: "Web design",
    testo: "Progettiamo siti semplici e piacevoli da usare."
  },
  consulenza: {
    titolo: "Consulenza",
    testo: "Ti aiutiamo a scegliere gli strumenti giusti."
  }
};

function Servizi() {
  return (
    <main>
      <h1>Servizi</h1>
      <p>Scegli un servizio per vedere i dettagli.</p>
      <Link to="/servizi/web-design">Web design</Link>
      <Link to="/servizi/consulenza">Consulenza</Link>
    </main>
  );
}

function DettaglioServizio() {
  const { serviceId } = useParams();
  const servizio = servizi[serviceId];

  if (!servizio) {
    return <main><h1>Servizio non trovato</h1><Link to="/servizi">Torna ai servizi</Link>
  </main>;
  }

  return <main><h1>{servizio.titolo}</h1><p>{servizio.testo}</p></main>;
}

<Route path="servizi/:serviceId" Component={DettaglioServizio} />
```